Task 1: player wins:

```
/Users/maahighotra/PycharmPro
PLAYER TURN
Select move...

Invalid move. Try again.
Select move...
1
Invalid move. Try again.
Select move...
In [1]
Invalid move. Try again.
Select move...
normal
You selected:  normal
Player health: 100
Enemy health: 80

ENEMY TURN
Enemy selected:  special
Player health: 90
Enemy health: 85

PLAYER TURN
Select move...
special
You selected:  special
Player health: 95
Enemy health: 75
```

```
ENEMY TURN
Enemy selected:  last stand
Player health: 65
Enemy health: 60

PLAYER TURN
Select move...
heal
You selected:  heal
Player health: 80
Enemy health: 60

ENEMY TURN
Enemy selected:  heal
Player health: 80
Enemy health: 75

PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 65
Enemy health: 45

ENEMY TURN
Enemy selected:  normal
Player health: 45
Enemy health: 45
```

```
PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 45
Enemy health: 25

ENEMY TURN
Enemy selected:  heal
Player health: 45
Enemy health: 40

PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 45
Enemy health: 20

ENEMY TURN
Enemy selected:  last stand
Player health: 15
Enemy health: 5

PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 15
Enemy health: -15

GAME OVER: Player wins.

Process finished with exit code 0
```

Enemy wins:

```
/Users/maahighotra/PycharmProjects/PythonPr
PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 100
Enemy health: 80

ENEMY TURN
Enemy selected:  normal
Player health: 80
Enemy health: 80

PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 65
Enemy health: 50

ENEMY TURN
Enemy selected:  last stand
Player health: 35
Enemy health: 35

PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 20
Enemy health: 5

ENEMY TURN
Enemy selected:  normal
Player health: 0
```

```
ENEMY TURN
Enemy selected:  normal
Player health: 0
Enemy health: 5

GAME OVER: Enemy wins.

Process finished with exit code 0
```

Draw:

```
/Users/maahighotra/PycharmProjects/Python
PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 85
Enemy health: 70

ENEMY TURN
Enemy selected:  last stand
Player health: 55
Enemy health: 55

PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 40
Enemy health: 25

ENEMY TURN
Enemy selected:  last stand
Player health: 10
Enemy health: 10

PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: -5
Enemy health: -20

DRAW.

Process finished with exit code 0
```

Task 2: Before Changing
List size: 1000000
Target: rand int (0, 999999)

```
/Users/maahighotra/PycharmProjects/PythonProject/.venv/bin/python /
99999
-1
1
8
Linear search execution time: 0.839941 seconds
Binary search execution time: 0.000294 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.880301 seconds
Insertion Sort Execution Time: 1.593647 seconds
Binary Search (With Sorting) Execution Time: 1.492856 seconds
Linear Search Execution Time: 0.000284 seconds

Process finished with exit code 0
```

After Changing:
List Size: 1000
Target: 70

```
/Users/maahighotra/PycharmProjects/PythonProject/.venv/bin/python /U
99999
-1
1
8
Linear search execution time: 0.000142 seconds
Binary search execution time: 0.000118 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.917424 seconds
Insertion Sort Execution Time: 1.611480 seconds
Binary Search (With Sorting) Execution Time: 1.500081 seconds
Linear Search Execution Time: 0.000042 seconds

Process finished with exit code 0
```

After changing the sorted list to 1000 from 1000000 and changing the target from random integer to 70 which was a number in the range 0 - 999999, there was a slower time in execution by milliseconds. Although the execution time with sorting was faster which might be because there was a given target instead of having a random integer.

Task 3:
Before Changing:

```
/Users/maahighotra/PycharmProjects/PythonProject/.venv/bin/python /Use
999
-1
1
8
Linear search execution time: 0.000143 seconds
Binary search execution time: 0.000269 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.983735 seconds
Insertion Sort Execution Time: 1.652316 seconds
Binary Search (With Sorting) Execution Time: 1.523288 seconds
Linear Search Execution Time: 0.000028 seconds

Process finished with exit code 0
```

```
/Users/maahighotra/PycharmProjects/PythonProject/.venv/bin/python /U
999
-1
1
8
Linear search execution time: 0.000143 seconds
Binary search execution time: 0.000272 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.982459 seconds
Insertion Sort Execution Time: 1.616453 seconds
Binary Search (With Sorting) Execution Time: 1.520024 seconds
Linear Search Execution Time: 0.000032 seconds

Process finished with exit code 0
```

After changing the random integer from 1 to 1000 to 1 to 34, and also changing the in range to 700 from 10000, there was a slight decrease in time and the bubble sort execution was quicker. This shows that reducing range leads to a faster sorting.

Task 4:

```
99999
-1
1
8
Linear search execution time: 0.000136 seconds
Binary search execution time: 0.000090 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 0.019734 seconds
Insertion Sort Execution Time: 0.010410 seconds
Binary Search (With Sorting) Execution Time: 1.454208 seconds
Linear Search Execution Time: 0.000129 seconds

Data size:  100
Insertion sort + linear search:  0.00013037491589784622
Bubble sort + linear search:  0.0002195840934291482
Insertion sort + binary search:  0.00012070895172655582
Bubble sort + binary search:  0.0002119590062648058

Data size:  600
Insertion sort + linear search:  0.0045513330260291696
Bubble sort + linear search:  0.008522624964825809
Insertion sort + binary search:  0.004537541070021689
Bubble sort + binary search:  0.008664333960041404

Data size:  7000
Insertion sort + linear search:  0.6723587909946218
Bubble sort + linear search:  1.3331061670323834
Insertion sort + binary search:  0.6803237090352923
Bubble sort + binary search:  1.3387977079255506
```

Observation 1: The sorting algorithm is much more important than the search algorithm. In all three cases, bubble sort with anything is at least twice as slow as insertion sort with anything. This is because bubble sort is just doing a lot of extra work. Bubble sort is comparing and swapping values even when the list is nearly sorted. Insertion sort is much smarter than bubble sort.

Observation 2: The search algorithm is irrelevant. In all three cases, insertion+linear and insertion+binary take essentially the same amount of time. The same is true for bubble+linear and bubble+binary. The reason is that the search algorithm is just too quick. We spend all the time on the sorting algorithm.

Observation 3: Unsorted linear search beats all four combinations when we only need to search once. A simple linear search on an unsorted list takes around 0.0094 seconds on 7,000 elements. That's 70 times faster than the best combination of sort and search. This is

exactly what we found out in Task 2. If you only need to find one item, just scan the list. Sorting the list is a waste of time.

The winning combination is which, and why:

The combination of insertion sort + binary search is a winner by a very narrow margin over insertion sort + linear search. The point is that insertion sort is a better algorithm than sorting, and adding a fast search algorithm like binary search on top of it is a waste of time because sorting is still a slow process.

The combinations become worthwhile when you need to search a list many times. Then, sorting once with insertion sort, followed by using binary search to search every time, gives you a good combination because insertion sort is efficient during the initial sorting, and then all your searches are fast.